Assignment 1: The Great Bitwise Bake Off

Nallathambi Vethiappan, Ekansh Khanulia, Aparajita Saha
Computer Science Department
Leiden University

Abstract

This project presents a generative system that creates new cookie recipes using a *Simple PIERRE* inspired genetic algorithm. Recipes from a JSON knowledge base form the initial population and evolve through crossover and mutation. A fitness function evaluates each recipe for structure, ingredient diversity, and novelty, ensuring outputs remain coherent and creatively distinct. Over several generations, the system produces plausible yet imaginative flavour combinations. The top three evolved recipes are compiled into an AI-assisted cookbook. This work demonstrates how evolutionary search can support computational creativity by balancing randomness, constraint, and culinary knowledge.

Introduction

Computational creativity aims to design systems capable of producing artifacts that exhibit novelty, value, and coherence. Recipe generation provides a testbed for such creativity, where combinations must be both innovative and edible. Inspired by the PIERRE system, this project implements a genetic algorithm to evolve new cookie recipes from an inspiring set of existing ones.

Each recipe is represented as structured data containing ingredients, amounts, and roles. Through genetic operations such as crossover and mutation, the system recombines and modifies ingredients to generate novel recipes while maintaining essential baking structure. Fitness evaluation rewards recipes that achieve balance, uniqueness, and plausibility. By iteratively refining the population across generations, the system demonstrates how evolutionary computation can facilitate creative exploration in the culinary domain, producing diverse and well-structured cookie recipes suitable for presentation in a generative cookbook.

Knowledge Base Construction

The knowledge base comprises 17 cookie recipes in a custom JSON format, each annotated with metadata—name, category, flavour and texture profiles, meal type, and ingredients grouped as base, add-ins, or toppings. Ingredient entries specify amount, unit, role, and type, enabling flexible recombination.

To foster creativity, the set includes both sweet and savory recipes with diverse flavour profiles and ingredient types. This variety supports the genetic algorithm in generating novel and plausible combinations.

The schema draws on open recipe formats and datasets (Papathanasiou 2019) (Cochran 2021), informing metadata fields and ingredient roles to guide creative and valid outputs.

Methodology and Implementation

This section describes the design and implementation of a genetic algorithm system for generating novel cookie recipes.

Recipe Representation

All recipes, whether from the knowledge base or generated during evolution, are represented as instances of the `CookieRecipe` class. This unified structure supports efficient manipulation and recombination. During processing, duplicate ingredients are merged by summing their amounts, and toppings are capped at the three most abundant to maintain a balanced search space and avoid excessive complexity.

Recipe Embedding and Text Preprocessing

Ingredients from all recipe sections are concatenated into descriptive text strings. To improve the quality of semantic embedding, common stopwords and generic cooking terms (e.g., ”flour”, ”egg”) are filtered out. The SentenceTransformer model `all-MiniLM-L6-v2` is used to embed these descriptions into high-dimensional feature vectors that capture semantic similarity.

The `all-MiniLM-L6-v2` model was selected due to its ability to generate compact and semantically meaningful embeddings from short texts. This model balances accuracy and computational efficiency, making it well-suited for embedding ingredient lists repeatedly during genetic algorithm optimisation.

Fitness Function Design

The fitness function quantitatively evaluates each candidate recipe to guide the evolutionary process towards producing valid, diverse, and novel recipes. It incorporates:

- **Structural Validity:** Severe penalty (-10) for missing essential roles (structure, fat, binder, leavening).
- **Ingredient Diversity:** Reward for unique normalized ingredients.
- **Ingredient Distribution:** Positive scores for recipes with about three base ingredients, 2-6 flavour add-ins, and at least one topping.
- **Novelty:** One minus the maximum cosine similarity between a candidate and all original recipes, encouraging semantic difference.
- **Combined Score:** Rounded to two decimal places to ensure numerical stability and consistent precision.

These fitness criteria were selected to ensure the generated recipes uphold common-sense baking principles. Specifically, the presence of essential base ingredients is critical; without structure, fat, binder, and leavening components, cookies cannot be properly baked. The severe penalty for missing any of these bases prevents the generation of non-viable recipes. Ingredient diversity encourages exploration of novel ingredient combinations without redundancy. The ingredient distribution preferences reflect typical cookie recipe composition, preserving culinary balance. Novelty based on semantic embedding similarity fosters creativity by encouraging the generation of recipes differing meaningfully from the original set.

Genetic Algorithm Operators

The algorithm uses crossover and mutation to generate new recipes by recombining and altering ingredient lists.

Crossover Crossover combines parts of two parent recipes to make a new recipe. To keep recipes logical, crossover happens separately for each ingredient category: base ingredients, flavour add-ins, and toppings. If both parents have more than one ingredient in a category, a random cut point splits the ingredients. The new recipe gets some ingredients from one parent and the rest from the other. Sometimes, with a 30% chance, the entire category is copied from one parent. This keeps ingredient groups clear and avoids mixing different types (Obitko 1997).

Mutation Mutation changes recipes in different ways: adding, removing, replacing ingredients, or adjusting amounts. Amounts are adjusted by multiplying by a random number between 0.8 and 1.3. Mutation happens with a 60% chance per new recipe. When adding flavour ingredients, rare ingredients that appear less often in the population are chosen more often to encourage novelty. Mutation tries up to five times each time it runs to avoid too many random changes. The ingredient group to mutate is chosen randomly among base ingredients, flavour add-ins, and toppings (Wikipedia contributors 2002).

Algorithm Overview

The main steps of the genetic recipe generation process are outlined in Algorithm 1.

Algorithm 1 Genetic Cookie Recipe Generator

Require: Recipe dataset R (JSON)
Ensure: Top 3 novel cookie recipes generated

procedure MAIN
  $R \leftarrow$ LOADRECIPES($'recipes.json'$); $P \leftarrow$ random initialization from R
  for $g \leftarrow 1$ **to** G **do**
    for each recipe $r \in P$ **do**
      $fit[r] \leftarrow$ EVALUATEFITNESS(r)
    end for
    $P' \leftarrow \emptyset$
    while $|P'| < N$ **do**
      $p_1, p_2 \leftarrow$ SELECTPARENTS(P, fit)
      $child \leftarrow$ CROSSOVER(p_1, p_2)
      if $Random() < \mu$ **then**
        MUTATE($child$)
      end if
      $P' \leftarrow P' \cup \{child\}$
    end while
    $P \leftarrow P'$
  end for
end procedure
procedure CROSSOVER(p_1, p_2)
  $k \leftarrow$ random cut index
  $child.dough \leftarrow$ mix of first k from p_1 and remainder from p_2
  $child.exterior \leftarrow$ random mix from both parents
  return $child$
end procedure
procedure MUTATE(r)
  Randomly apply one of:
    adjust amount, replace, add, or remove ingredient
  Normalize duplicates and units
end procedure

Population Initialization and Evolution

The evolutionary process begins with an initial population of 20 recipes per generation. One-third of these are original recipes from the knowledge base, while the remaining two-thirds are mutants created by applying mutations to the original recipes. The population evolves over eight generations.

In each generation, recipes are evaluated using the fitness function. Parents are selected for reproduction with probabilities weighted by their fitness scores. New offspring are produced through crossover and mutation operations. This process repeats, with offspring replacing the current population, allowing the system to progressively explore and refine recipe variations.

Selection of Diverse Recipes

Three recipes are selected for diversity and high fitness, using pairwise cosine embedding distance to ensure semantic variety. If no recipe meets the fitness threshold, the system lowers it to include more options.

Recipe Naming Strategy

Recipe names are generated from the main flavour add-ins and toppings, with stopwords and extra descriptors removed. Up to three unique, high-quantity ingredients are prioritized, and names always end with "Cookie." This approach ensures names are descriptive and relevant.

Experimental Setup

Experiments were implemented in Python 3 using scikit-learn, SentenceTransformer, NLTK, and matplotlib libraries. Key experimental parameters are summarized in Table 1. Data sources and preprocessing steps are described in the Methodology and Implementation section.

Table 1: Key Experimental Parameters

Parameter	Value
Population size	20
Number of generations	8
Mutation rate	0.6
Fitness threshold	3
Crossover probability	0.3
Max mutation attempts	5
Embedding model	all-MiniLM-L6-v2
Selection method	Fitness + Diversity

Each run starts with a mix of original and mutated recipes. The genetic algorithm proceeds through fitness evaluation, parent selection, crossover, mutation, and replacement for eight generations. At the end, three diverse and high-fitness recipes are selected for analysis.

Due to the stochastic nature of the algorithm, the exact recipes generated differ with each run. This approach is intended to explore a wide range of novel solutions rather than reproduce identical outputs.

Quantitative Evaluation of Creativity

To assess the creative performance of our system, we evaluate the generated recipes using several quantitative and visualization-based metrics. These measures help determine whether the system produces outputs that are both novel and diverse.

Diversity and Overlap Metrics

- **Flavour Diversity:** The average pairwise Jaccard distance for major flavour add-ins shows how different the selected recipes are.
- **Ingredient Overlap:** The average pairwise Jaccard similarity for all ingredients indicates the uniqueness of ingredient combinations.

Fitness Distribution

Fitness scores are grouped into high, medium, and low categories to summarise the overall quality of the final population.

Embedding Visualization

Recipe descriptions are embedded using the SentenceTransformer model and reduced to two dimensions with PCA. A scatter plot displays original and generated recipes, showing clustering and diversity.

These evaluation metrics provide insight into the novelty, diversity, and quality of the system's outputs.

Results

This section reports the creativity evaluation metrics and visual outcomes achieved by the recipe generation algorithm.

Population Statistics

A total of 160 recipes were generated over eight generations. In the final population:

- 65% (13 recipes) achieved high fitness (≥ 7)
- 25% (5 recipes) had medium fitness (3–7)
- 10% (2 recipes) had low fitness (< 3)

Top Generated Recipes

The system selected three diverse, high-scoring recipes:

1. **Apples Cheese Cheddar Cookie**

   Fitness Score: 9.34

   Base Ingredients: all-purpose flour, butter, baking powder, egg

   Flavour Add-ins: instant coffee, brown sugar, salt, grated apples, cinnamon powder, grated cheddar cheese

   Toppings: grated cheddar, chocolate drizzle, toasted coconut flakes

2. **Blueberry Sesame Pistachios Cookie**

   Fitness Score: 8.22

   Base Ingredients: all-purpose flour, olive oil, baking powder, egg.

   Flavour Add-ins: blueberry puree, crushed pistachios, toasted sesame seeds, ginger powder

   Toppings: sea salt flakes, chocolate drizzle, white sesame seeds

3. **Jalapenos Almonds Brown Cookie**

   Fitness Score: 7.40

   Base Ingredients: all-purpose flour, butter, baking powder, egg

   Flavour Add-ins: dried lavender, brown sugar, diced jalapenos, vanilla extract

   Toppings: crushed almonds, lavender sugar

The diversity of the top recipes is demonstrated by an average pairwise Jaccard distance of 1.000 for major flavour add-ins, indicating no overlap among the selected recipes. Ingredient uniqueness is also high, as shown by an average pairwise Jaccard similarity of 0.208 for all ingredients.

Embedding Visualization

Figure 1 shows a PCA scatter plot of the recipe embedding space. Original recipes are displayed with blue circles, and three generated recipes with red crosses. The generated recipes are visibly separated from the original cluster, indicating both semantic novelty and diversity.

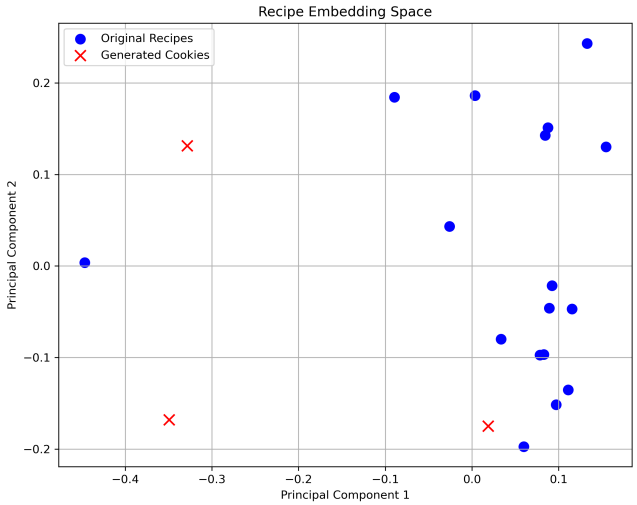

Figure 1: PCA plot of original and generated recipes

Cookbook Generation and Presentation

The final recipes were formatted using a Canva template, with names and ingredients taken directly from the algorithm's output. For visual presentation, images were generated using Gemini-AI. The process was straightforward—high-resolution, creative images were produced with just a few prompts, and the visuals effectively reflected the unique qualities of each recipe. Using Generative AI enhanced the cookbook's appeal and creativity, making it easy to achieve consistent and attractive results.

Reflection and Evaluation

The results show that the genetic algorithm consistently produces recipes with high fitness and good diversity. The top recipes have high fitness scores and distinct flavour profiles and ingredient combinations, as confirmed by the Jaccard metrics. The PCA visualisation also shows that the generated recipes are well separated from the original ones in embedding space.

However, there are some limitations. The current model replaces ingredients without always applying common culinary logic or ensuring essential base components are included. This can sometimes lead to less practical recipes. The system depends on a rich and well-structured JSON dataset; incomplete or poorly formatted data can reduce recipe quality. Careful tuning of parameters is also needed to balance diversity and validity. Since the algorithm is stochastic, results vary between runs and specific recipes cannot be exactly reproduced.

The evaluation uses quantitative metrics and semantic embeddings, which may not fully capture subjective aspects like taste or creativity. Future work could include human evaluation, expand the dataset, and explore better embedding models and ingredient logic to improve creativity assessment.

Beyond quantitative measures, the generated recipes often combined ingredients in unexpected and imaginative ways. Several outputs featured novel pairings—such as savory elements with sweet bases or unusual spice blends—that were genuinely surprising and would not have been anticipated manually. This highlights the system's capacity for creative exploration and suggests that algorithmic approaches can yield inspiring results in the culinary domain.

Overall, the system shows promise for computational creativity in recipe generation, but there is room for improvement.

Conclusion

This project demonstrates that a genetic algorithm, combined with embedding-based evaluation, can generate creative and diverse cookie recipes. The system produced high-fitness, novel recipes with distinct ingredient profiles, as shown by both quantitative metrics and visualization. While the approach has limitations—such as reliance on structured data, the need for careful tuning, and occasional lack of culinary logic—it provides a strong foundation for computational creativity in recipe generation. Future improvements could include more advanced ingredient logic, larger datasets, and human evaluation to further enhance creativity and practicality.

References

[Cochran 2021] Cochran, M. 2021. json-cookbook: A collection of cooking recipes in json format. https://github.com/micahcochran/json-cookbook. Accessed: 2025-10-15.

[Obitko 1997] Obitko, M. 1997. Crossover and mutation - introduction to genetic algorithms. https://www.obitko.com/tutorials/genetic-algorithms/crossover-mutation.php. Accessed: 2025-10-15.

[Papathanasiou 2019] Papathanasiou, D. 2019. A collection of cooking recipes in json format. https://github.com/dpapathanasiou/recipes. Accessed: 2025-10-15.

[Wikipedia contributors 2002] Wikipedia contributors. 2002. Genetic algorithm. https://en.wikipedia.org/wiki/Genetic_algorithm.